Decision log

The important choices, in roughly the order I made them: what I picked, what else I considered, and why.

Data

- **Removed 30 exact-duplicate rows and dropped the single `IN_TRANSIT` order. Normalised `Rto` / `delivered` / `RTO`.** *Alternative:* keep the rows as-is. *Why:* duplicates double-count outcomes, and an in-transit order has no label to learn from. The raw status spellings first showed up as separate 0%-RTO categories in my exploration, which is how I noticed them.
- **Kept zero-value orders (3), 9-digit phones (17) and missing pincodes (12) rather than deleting them.** *Why:* these situations will happen in production too, so the engine has to handle them, and the back-test should include them.
- **Capped `customer_prior_rto` at `customer_prior_orders` (4 rows).** You can't refuse more orders than you placed, so I treated it as a data error rather than a signal.

Model

- **Out-of-time split: train on Apr–Aug, test on September.** *Alternative:* a random 80/20 split. *Why:* the real question is "would this have worked next month?", and September is the "last month" the brief asks about. A random split would let late-September patterns leak into training.
- **Logistic regression, written in pure Python.** *Alternatives:* gradient-boosted trees, or an LLM scoring the whole order. *Why:* 2,000 rows and ~13 features don't need more. Every coefficient becomes one plain-English sentence, which the brief requires. The probabilities came out calibrated well enough for ₹ maths (September: predicted 57% vs actual 54% in the top fifth). Pure Python means the reviewer installs nothing. Test AUC is 0.754, against 0.752 in training, so no overfitting.
- **Area risk at pincode-district level (first 3 digits), smoothed toward the state rate (k=20), computed out-of-fold.** *Alternative:* per-pincode history, which is what Ops asked for. *Why:* 554 of 1,066 pincodes have exactly one order, so pincode-level "bad history" is mostly noise. Out-of-fold encoding stops a row's own outcome from leaking into its feature.
- **Left courier, hour of day and day of week out of the model.** Courier is assigned after this decision, and outside the 4 high-RTO states the couriers sit within 20–25% of each other. Time-of-day differences looked like noise at this sample size.

Where the LLM goes

- **The LLM does three jobs only:** address usability, name plausibility, and free-text WhatsApp replies. Everything numeric is code. *Alternative considered:* let the LLM read the whole order and decide. Rejected because it can't produce calibrated probabilities, its decisions can't be explained in ₹, it costs money on every order, and it can't be back-tested deterministically.
- **The LLM output is a model feature, never the decision itself.** A wrong LLM judgement shifts the risk score; it can't cancel an order by itself, apart from the "fake name AND junk address" rule.
- **Rule-based fallback for every LLM job, plus an on-disk cache and a circuit breaker after 3 failures.** *Why:* the system must keep shipping orders when the API is down or the key is missing, and the reviewer must be able to run it in under 10 minutes without a key. All memo numbers use the fallback, since no key was available during the build. The fallback is tuned to this dataset's address patterns; that's exactly why production should use the LLM, which generalises to real messy addresses.
- **The reason text comes from templates, not the LLM.** An LLM would write nicer prose, but it could state a reason the model didn't actually use. For an ops audit trail, correct beats pretty.
- **Model: `claude-opus-5-5` (in config).** Rough cost: about ₹0.40 per order, or about ₹700 a month at 1,650 COD orders, before caching repeat addresses. That's small next to one RTO (₹169). `claude-haiku-5-5` would cost roughly 1/40th as much and is likely enough for these classification jobs. It's a one-line config change, but it should be measured on a labelled sample first; I didn't switch it myself.
- **No server-side model fallback.** The local rule-based fallback already covers refusals, errors and timeouts for these jobs, so a second LLM hop adds cost without adding safety.

Economics (all in `rto/economics.py`, all inputs in config)

- **RTO cost = ₹169.4** (₹65 + ₹65 + ₹25 + 8% × ₹180).

- **A delivered order is worth 62% × order value − ₹65 forward shipping** (₹337 at the ₹649 average). *Ambiguity:* the brief doesn't say whether the 62% margin already includes shipping. I assumed it doesn't, which is the conservative reading for cancelling: it makes a delivered order look *less* valuable. The ratio of a ₹337 win to a ₹169 loss is what drives everything below.
- **Meta CAC (₹300) is not in the per-order maths.** *Why:* it's already spent when we decide whether to ship. What cancelling a genuine first-time buyer actually destroys is this order's margin plus their future orders. I estimated the future orders from the data: 25% of orders come from repeat customers, about 0.33 future orders per new buyer, × ~₹230 ≈ **₹75**. Marketing's ₹300 view is in the sensitivity table, and the recommendation doesn't change under it.
- **Cancelling pays only above ~67% risk (repeat buyers) or ~71% (first-time buyers).** This single fact rules out Ops' blanket cancellations.
- **Confirmation, nudge and hold outcomes are labelled as assumptions, not data:** confirmation catches 45% of would-be RTOs and loses 8% of genuine buyers; 15% accept the prepaid offer; people who accept are half as risky as their band; 10% of held customers pay online. They are editable in the Rules tab, and the memo shows the result when they're worse.
- **Saved and lost are split by how the order would have ended.** Saved = ₹ recovered on would-be RTOs; lost = ₹ given up on would-be deliveries. My first version mixed these, so a nudge could show negative "lost", which is unreadable for a founder. In the rewrite, the expected figure is exactly the probability-weighted mix of the two outcomes, and a unit test checks this.

Rules: chosen by back-test, not intuition

Each rule was checked by its ₹ effect on Apr–Aug and on September, using real outcomes.

- **Thresholds 25 / 50 / 75 (nudge / confirm / hold).** *Alternatives:* my first guess was 25 / 35 / 75. A grid search on Apr–Aug picked 20 / 60 / 70. *Why 25 / 50 / 75:* it's near the best in both periods (₹8.9k Apr–Aug, ₹3.1k September) and stays clearly positive (₹1.7k) if WhatsApp confirmation works worse than assumed, whereas 25 / 35 / 75 goes negative in that case. Confirmation is the step that can cancel real customers, so it should start narrow and widen only once it's measured.
- **"Score alone never cancels a first-time buyer; the most it can do is WhatsApp confirm."** This is Marketing's protection. It costs about ₹100 in September (₹3,093 vs ₹3,199), roughly ₹500 a month. I kept it: it's cheap insurance against the worst outcome Marketing described, and it's one dropdown to turn off.
- **Hold if fake name AND unusable address.** 76–83% RTO, positive ₹ in both periods. Kept.
- **Hold after 2+ past refusals.** 69–100% RTO, which is close to break-even. Kept because those customers are now being offered a pay-online link instead.
- **Dropped "confirm if the name looks fake".** Fake names with a real address returned 0 of 8 times; the address does the work.
- **Dropped "confirm if pincode missing".** 17–22% RTO, which is not risky, and the ₹ was negative.
- **Changed "hold if phone is invalid" to "ship, and flag for the courier".** The back-test showed these orders deliver about half the time, so holding them lost ₹485 in Apr–Aug and ₹980 in September. We can't WhatsApp a broken number anyway.
- **Changed "hold possible duplicates" to "confirm".** Holding the second copy of a likely double order lost ₹7,296 in Apr–Aug. Tightening the match (same phone, address and basket within 2 hours, instead of phone and items within 24h) didn't help: the flagged pairs really are double-submits made minutes apart, but in the history both copies were often delivered. So the customer is now asked ("you ordered this twice, want both?") instead of one copy being cancelled.
- **Hold ≠ dead order.** Held customers get a pay-online link. That turns some holds into prepaid sales, which also answers Marketing's "they'll never come back".

Prepaid discount

- **A flat ₹40 (~6% of the average order).** *Alternatives:* a % discount, ₹50, or a discount for everyone. *Why:* switching one buyer at risk p to prepaid is worth about (0.5p − 4%) × (₹412 + ₹169). That's under ₹20 at 15% risk, ₹49 at 25%, ₹79 at 35% and ₹122 at 50%. So ₹40 pays for itself from the 25% risk level where the nudge band starts. Offering it to low-risk buyers mostly discounts people who'd have paid anyway. A flat amount reads clearly in a WhatsApp message and caps the cost on big baskets. The acceptance rate at ₹40 is unknown, so ₹30 vs ₹50 is the first A/B test.

WhatsApp flow

- **Language by delivery state.** Hinglish (Roman script) for UP, Bihar, MP, Rajasthan, Delhi and other Hindi-belt states; English elsewhere. Replying "Hindi"/"English" switches it. *Alternative:* regional languages (Tamil, Bengali...). That needs approved templates and translations, so it's on the next-steps list.
- **No messages between 21:00 and 08:00.** Many orders arrive between midnight and 6am; those messages wait until 08:00, and so do reminders and cancellations.
- **Confirm flow:** first message → reminder at +4h → last call at +20h → cancel at +24h, with a pay-online link left open. **The last call is never skipped:** if it falls in quiet hours it goes out in the morning and the deadline moves with it. My first version silently skipped it; the simulator caught that.
- **Unclear reply:** re-ask once with buttons, and on the second unclear reply hand over to a person (flagged on the conversation). Questions go straight to a person.
- **The nudge never blocks an order for more than 6 hours.** One message, no reminders, then it ships as COD. "No" to a discount just means "I'll pay cash".
- **The customer confirmed but never sent a full address → ship anyway,** because the courier can call. Cancelling someone who said yes would be the worst outcome for Marketing.
- **Mock sender in WhatsApp Cloud API payload shape** (interactive reply buttons, max 3), so going live means replacing one `send()` method.

Product and engineering

- **Python standard library only (http.server, no framework, no npm).** *Why:* the brief asks for under 10 minutes to run, and the scale here (thousands of orders a month) doesn't need more. The only optional dependency is the `anthropic` SDK.
- **Thresholds live in `config/policy.json`, edited through the Rules tab with a September preview.** *Alternatives:* a Google Sheet, or YAML. *Why:* a non-technical person changes numbers in a form and sees the ₹ effect before saving; the JSON file remains the audit trail.
- **The dashboard replays September as "yesterday".** There's no live store, and September is the honest month because the model never saw it. Each ₹ figure is shown both as the live system would estimate it (from risk scores) and against what really happened, so the founder can see how far the estimate is off.
- **The trained model is committed** so the server starts instantly. The LLM cache is not committed (it can be regenerated, and it would contain customer addresses).